

实验2实验报告

实验目的

- 1. 学习 组合逻辑电路的设计方法和步骤，掌握 译码器、编码器等组合 部件的设计方法。
- 2. 掌握串行加法器 设计方法，理解减法和比较运算的实现方法。
- 3. 掌握汉明码校验电路的设计方法。
- 4. 掌握桶形移位器的设计方法。

实验内容

1. 译码器实验

实验内容

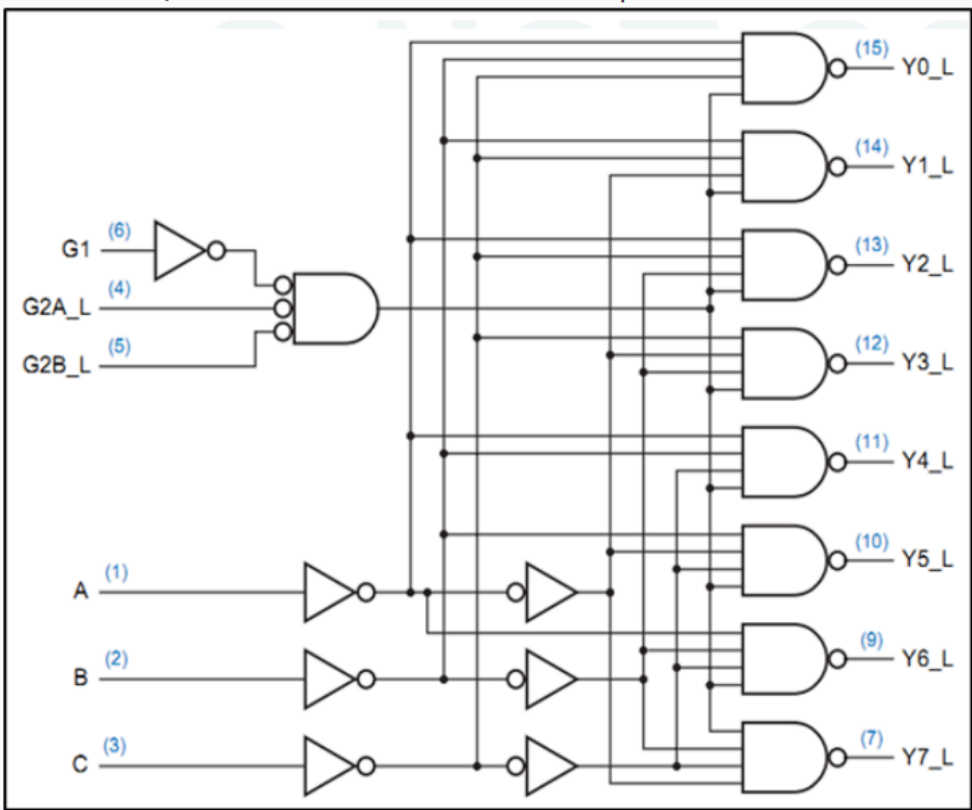
利用基本逻辑门设计一个3-8译码器

整体方案设计

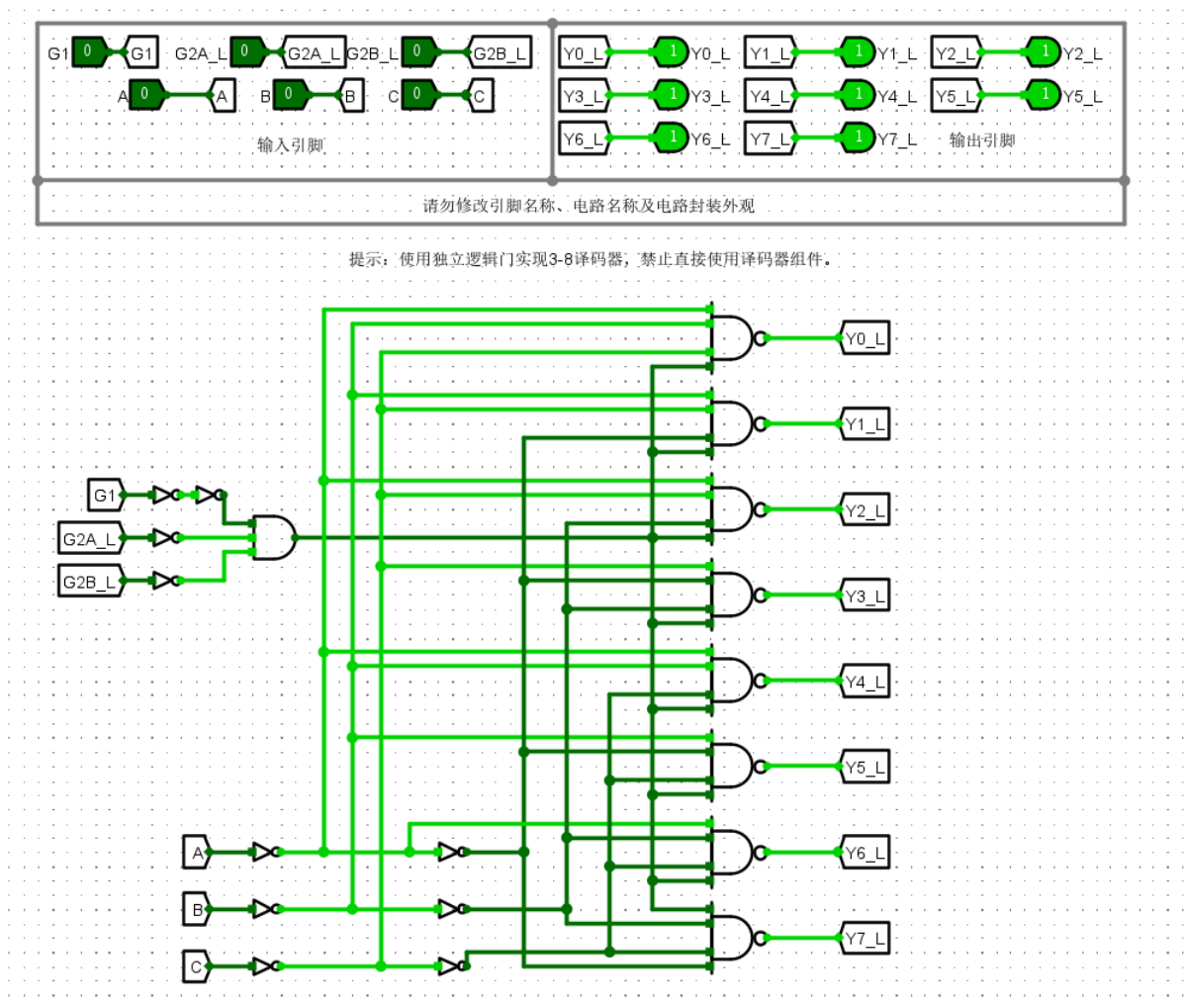
输入输出引脚：ABC代表了输入的三位数据，ABC是从低位到高位顺序，剩下三个引脚是使能端，当 $G1 * \overline{G2A_L} * \overline{G2B_L}$ 的逻辑运算结果为0时，不管ABC是什么值，都会输出全1。

原理图和电路图

原理图如下：

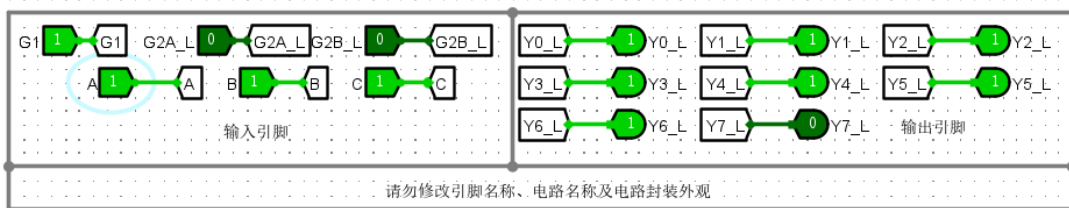


电路图如下：

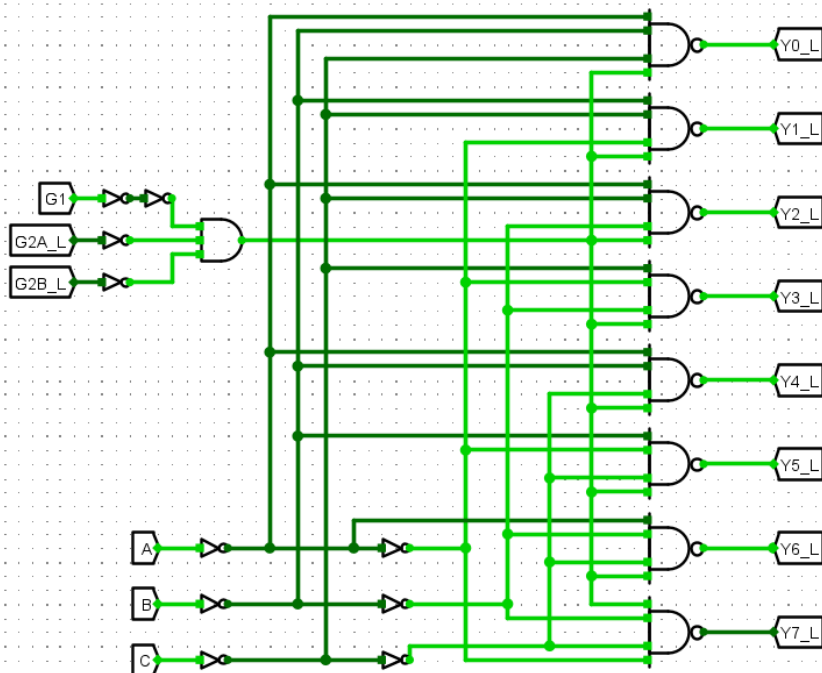


仿真测试图

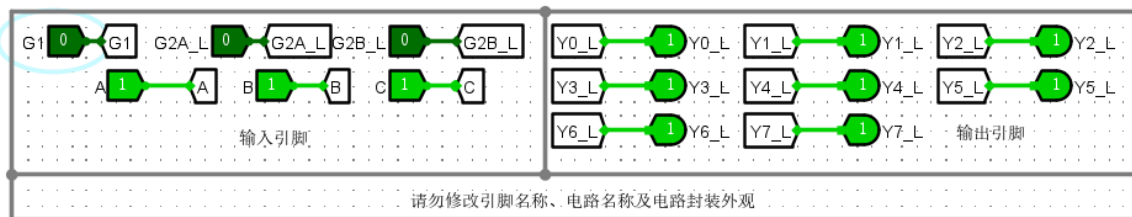
当使能端开放时，取 $A=B=C=1$ ，则 $Y7_L$ 输出0，其他输出1



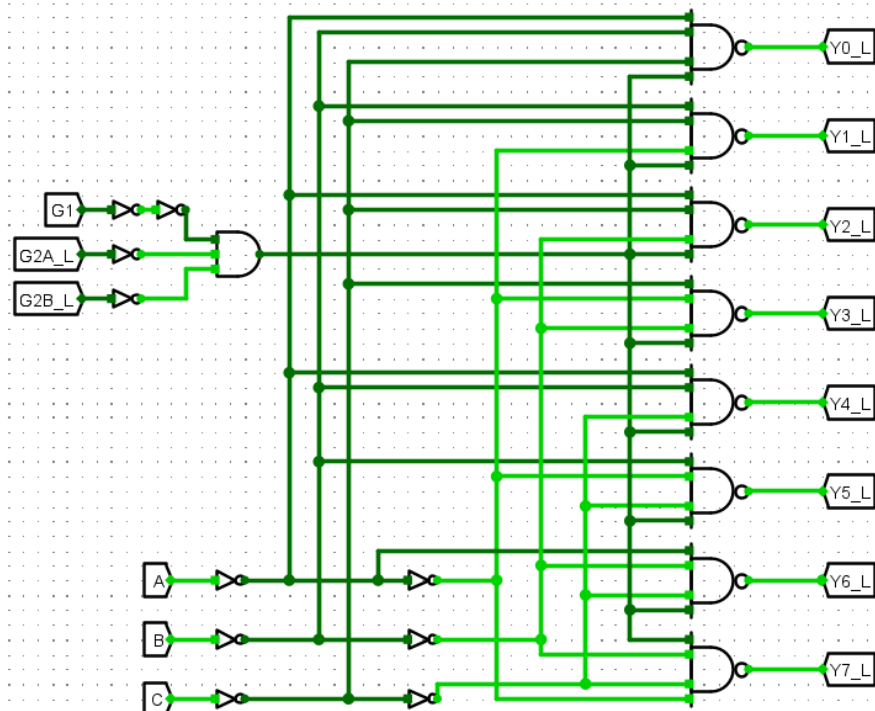
提示: 使用独立逻辑门实现3-8译码器, 禁止直接使用译码器组件。



当使能端取0,0,0时, 不管ABC如何取值, 都输出全1。



提示: 使用独立逻辑门实现3-8译码器, 禁止直接使用译码器组件。



真值表：

当 $G1=1$ ， $G2AL=G2BL=0$ 时，CBA代表的二进制数所对应的那个输出端输出0，其他输出1。

否则，全部输出1，不论CBA为多少。

错误分析

在完成实验的过程中，没有遇到任何错误。

2. 编码器实验

实验内容

根据优先级编码器原理图，设计一个由逻辑门电路构成的 8-3 优先级编码器。

整体方案设计

根据课上讲过的编码器逻辑表达式以及真值表和原理图进行实验设计

输入输出引脚：

I_0 - I_7 代表输入的数字，哪一个为1代表输入的数字是 $n+1$ 。

O_2 - O_0 代表输出输入数字的二进制表示，值得注意的是，在本实验中， $I_0=1$ 和 $I_0=0$ 的效果是一样的，都是输出全为0。

原理图和电路图

实验原理图如下：

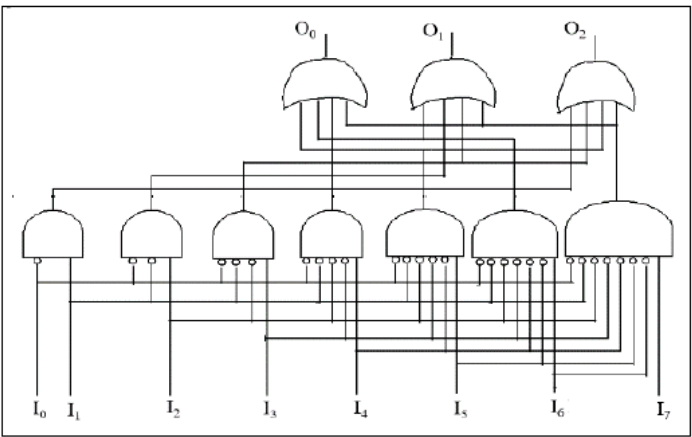
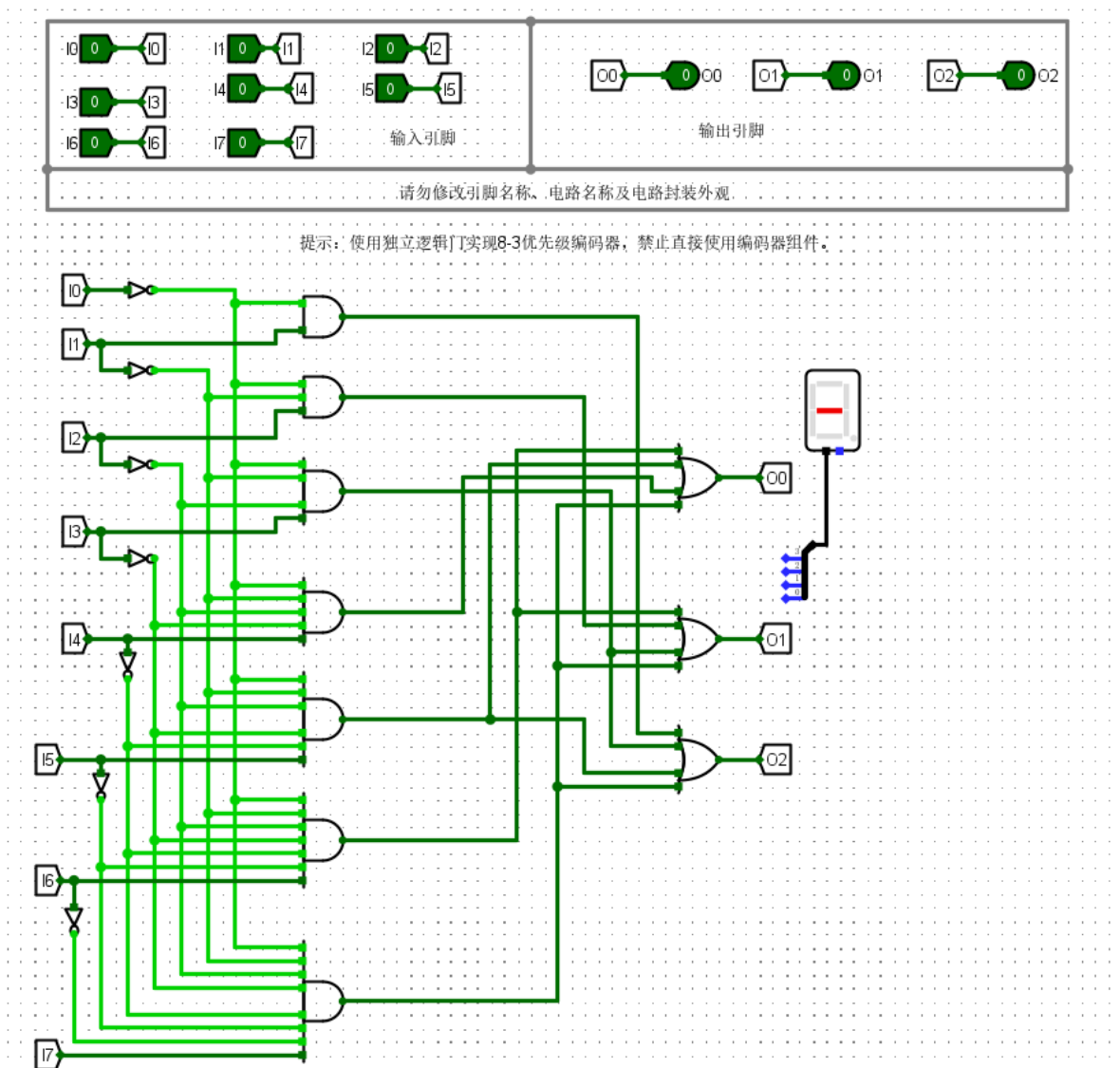


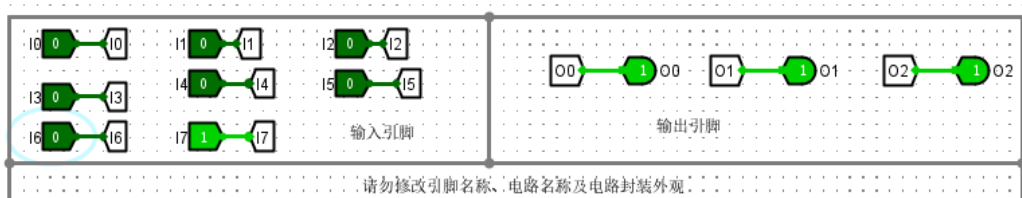
图 2.3 8-3 优先级编码器原理图

实验电路图如下：

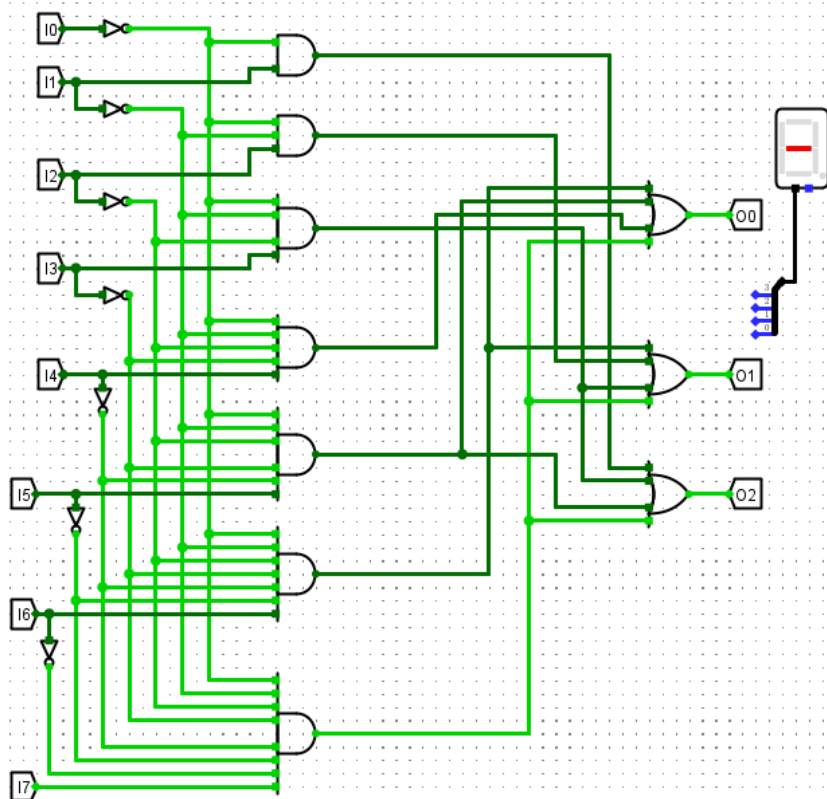


仿真测试图

选取输入数值为7和3两种情况进行测试：



提示：使用独立逻辑门实现8-3优先级编码器，禁止直接使用编码器组件。



错误分析

在完成实验的过程中，没有遇到任何错误。

3. 4位加法器实验

实验内容

利用全加器子电路，实现四位串行加法器

整体方案设计

大致思路是先设计好全加器，然后根据串行进位的加法原理，排列四位加法器进行设计，对于减法，则采取补码加法的原理，先按位取反，然后根据cin的值实现末位加一的效果。

输入输出引脚：

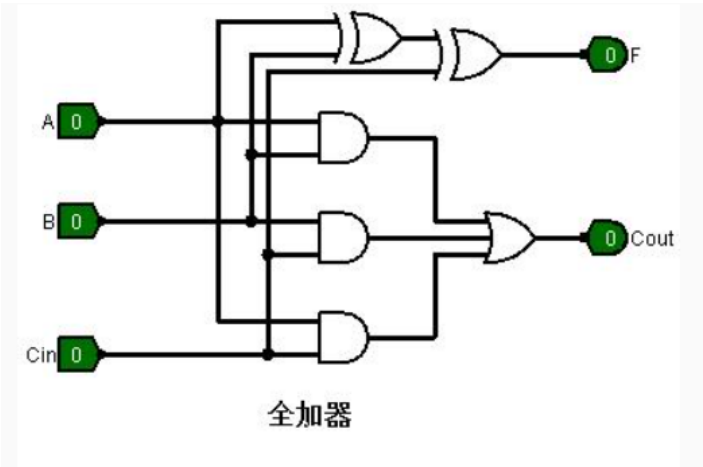
操作数X, Y：两个四位二进制数，代表要相加/减的两个数

cin：加减控制器，=0表示加，=1表示减。

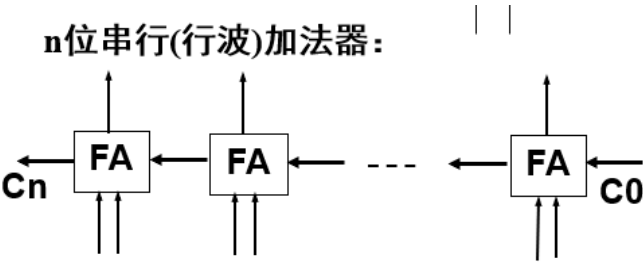
原理图和电路图

实验原理图如下：

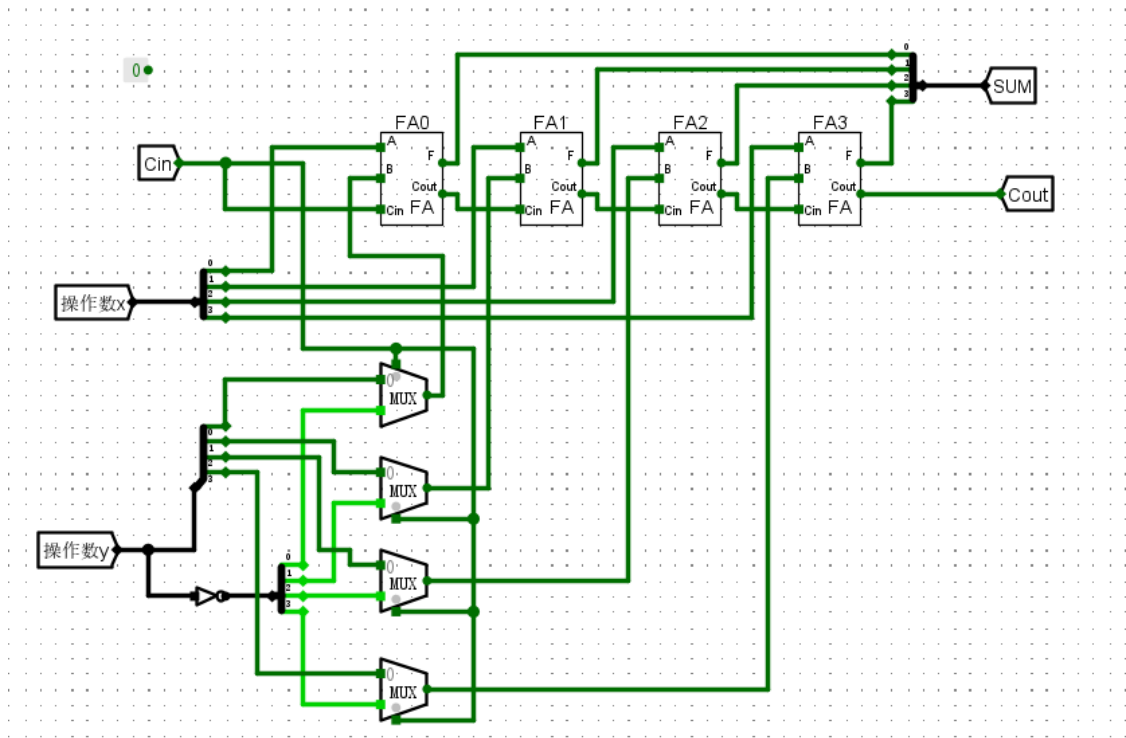
全加器原理图



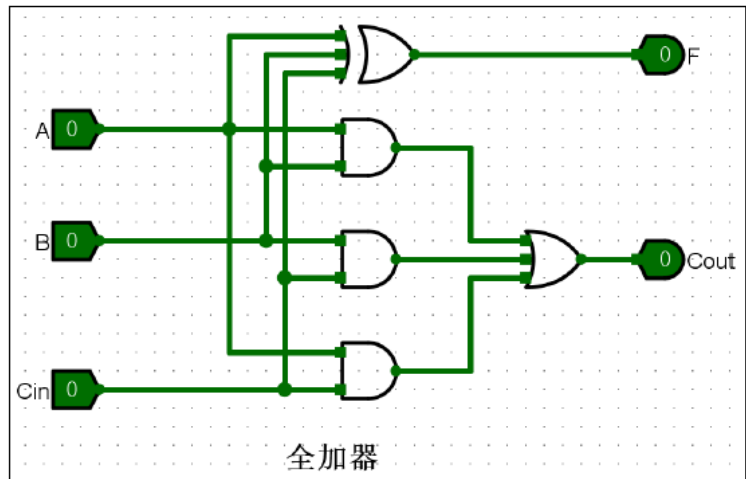
四位加法器原理图：



实验电路图如下：

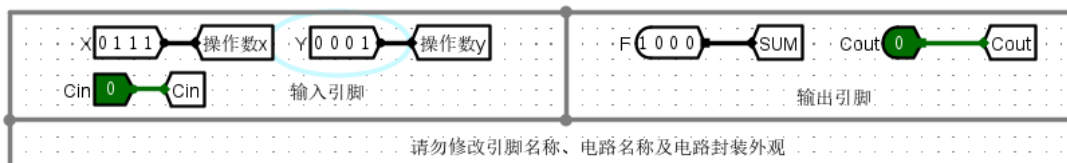


全加器子电路如下：

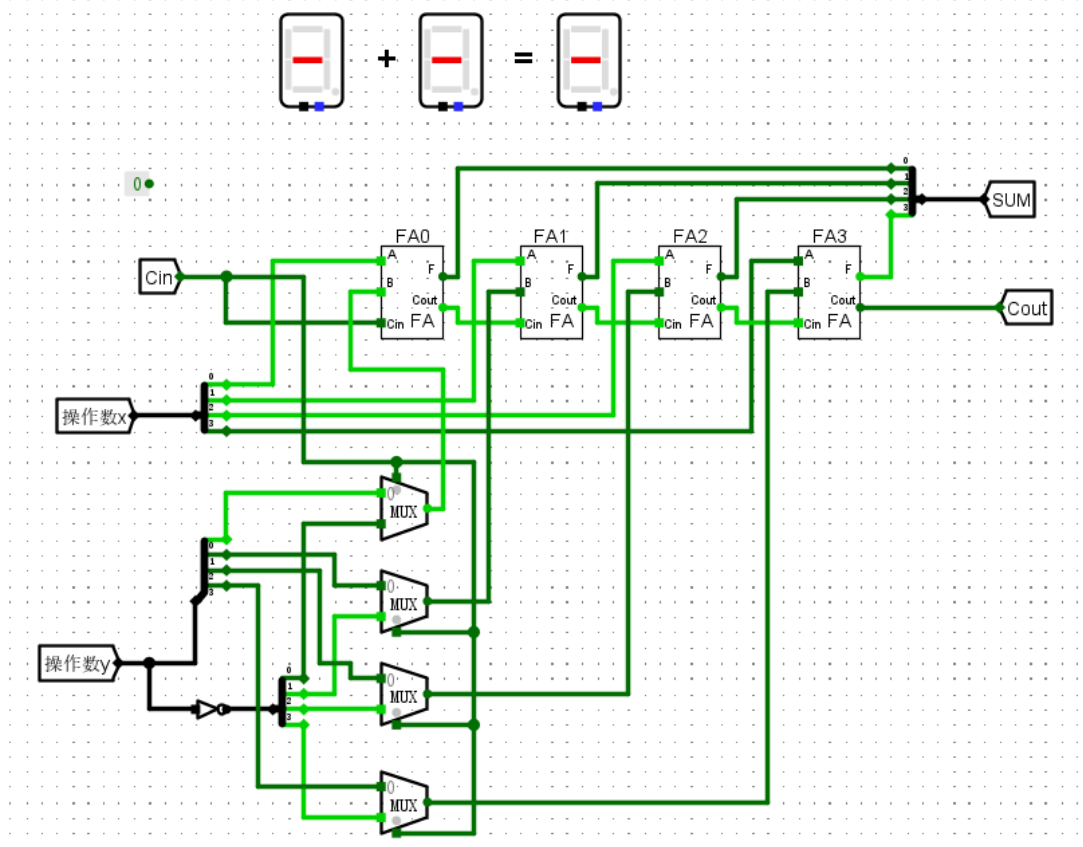


仿真测试图

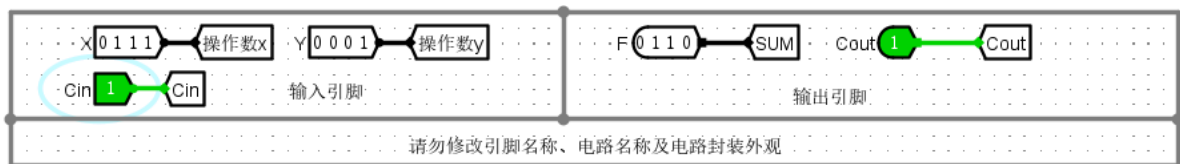
X=0111, Y=0001, cin=0时, 测试结果为1000, 正确:



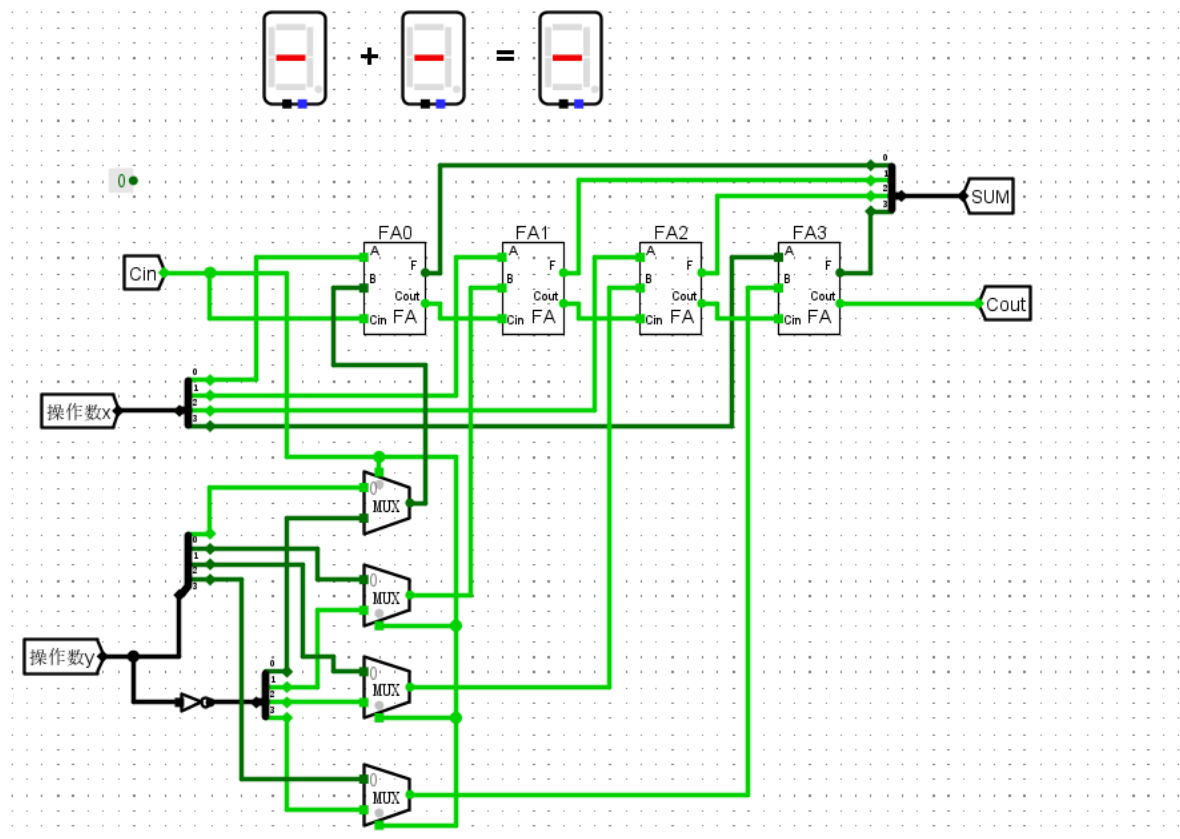
提示：使用独立逻辑门及全加器FA子电路实现4位二进制数加减法器，禁止直接使用加减法器组件。



X=0111, Y=0001, cin=1, 输出=0110, 正确:



提示：使用独立逻辑门及全加器FA子电路实现4位二进制数加减法器，禁止直接使用加减法器组件。



错误分析

在完成实验的过程中，没有遇到任何错误。

4. 汉明码纠错实验

实验内容

利用译码器、奇偶校验子电路、隧道和分线器等组件实现 7 位汉明码纠错功能，并显示发生错误位置和状态。

整体方案设计

思路是先设计好子模块——3-8译码器和4位奇偶校验器，注意这里的3-8译码器和实验1中的不完全一样，不可照搬。然后根据原理图，利用子模块设计即可。

输入输出引脚：

输入引脚：一个7位的二进制数，代表四位原始二进制数和3位校验位

输出引脚：7位数据，代表纠错后的数据。一个Noerror位，如果为0则证明有错误，否则证明无错误。

原理图和电路图

实验原理图：

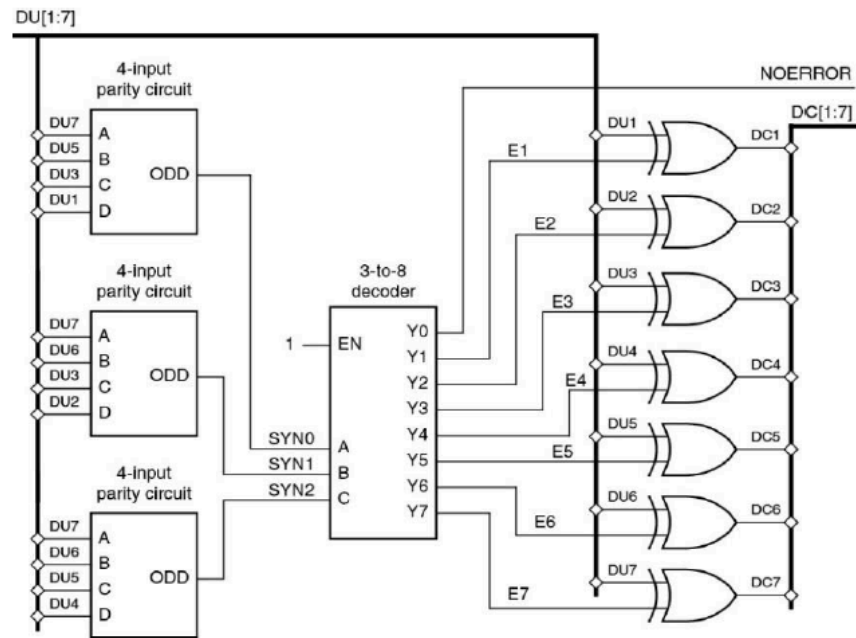
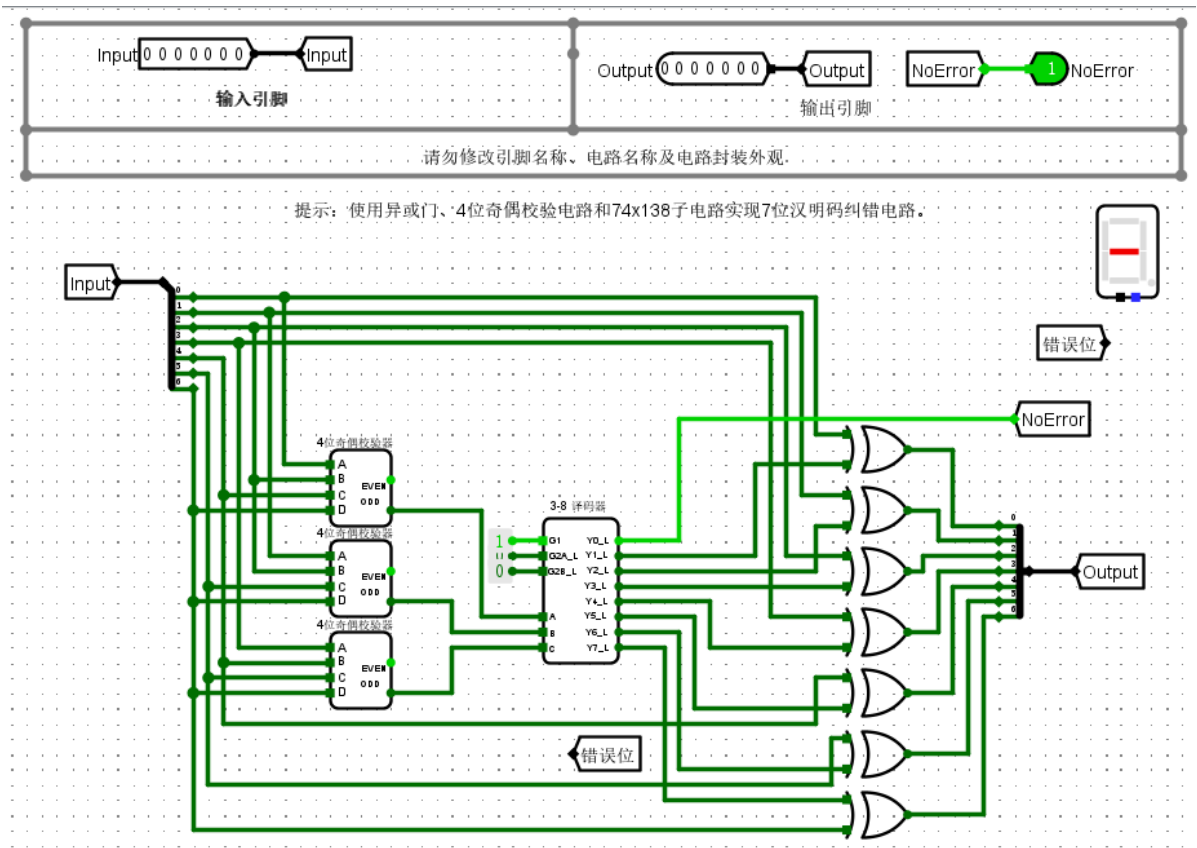
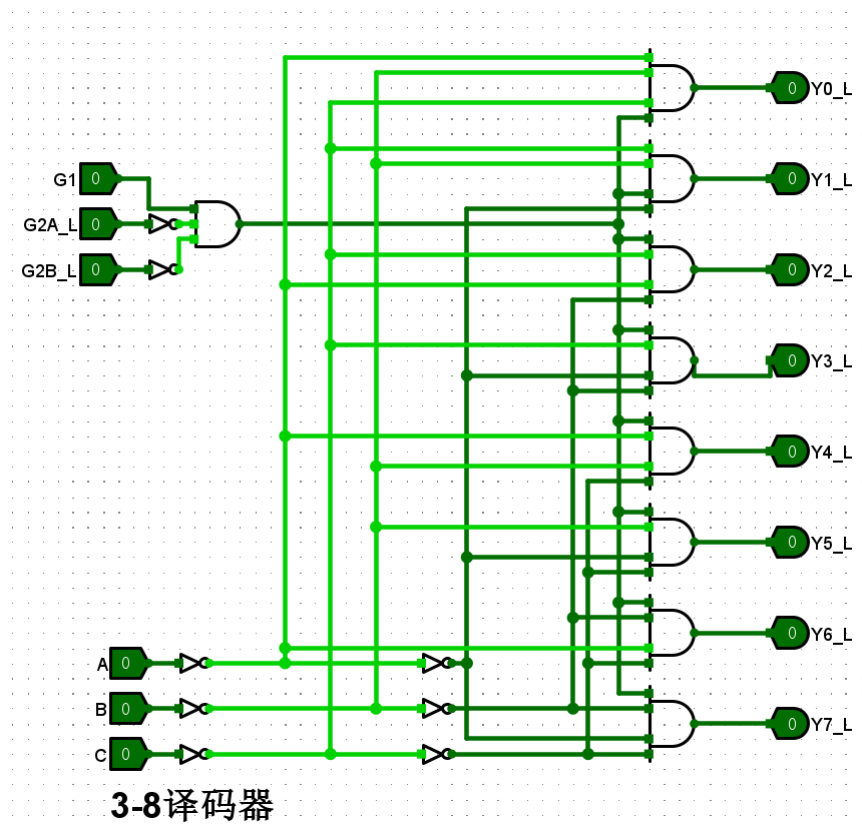


图 2.10 7 位汉明码检/纠错电路原理图

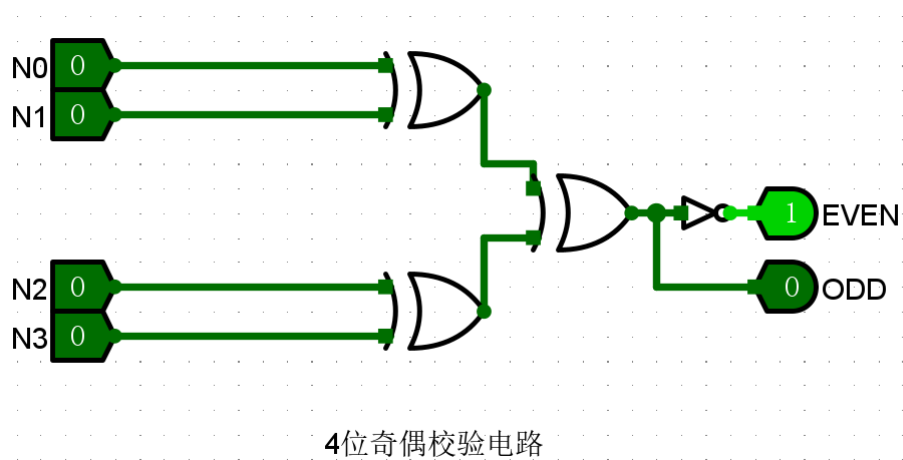
实验电路图：



3-8译码器电路图：

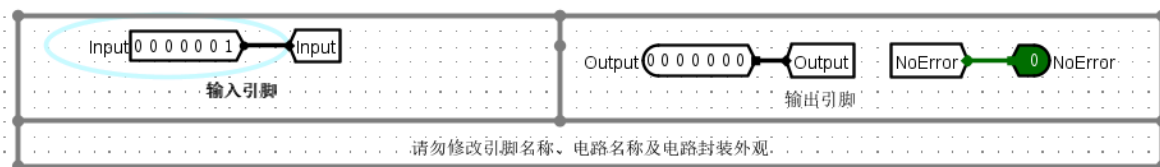


4位奇偶校验器电路图：

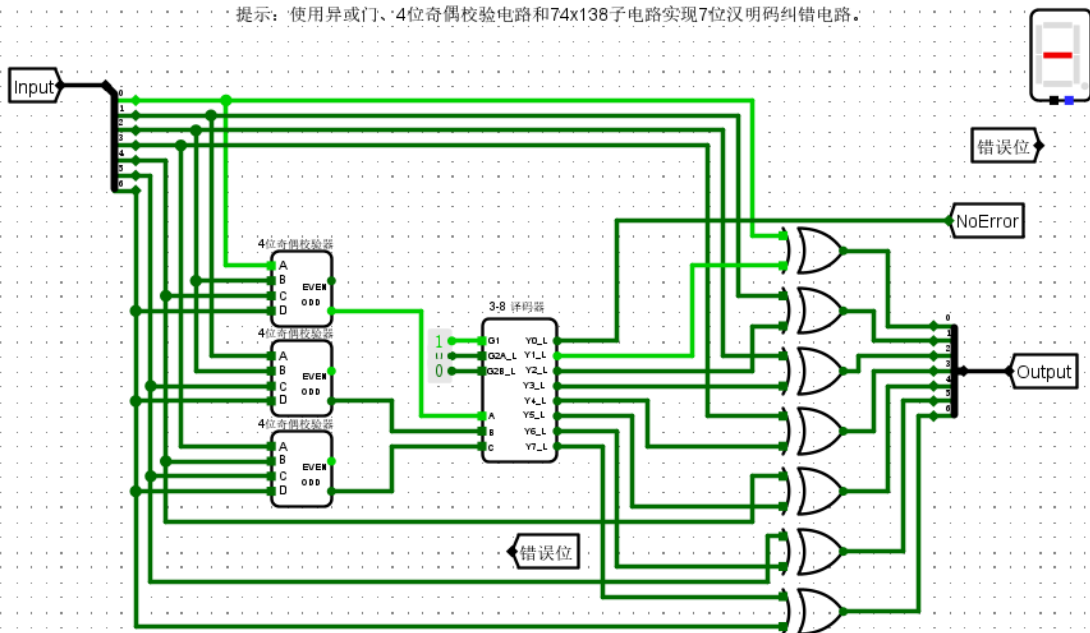


仿真测试图

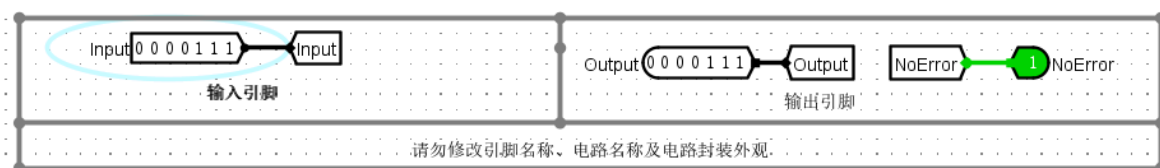
Input=0000001时，最后一位出错，恢复为全0，Noerror输出0.



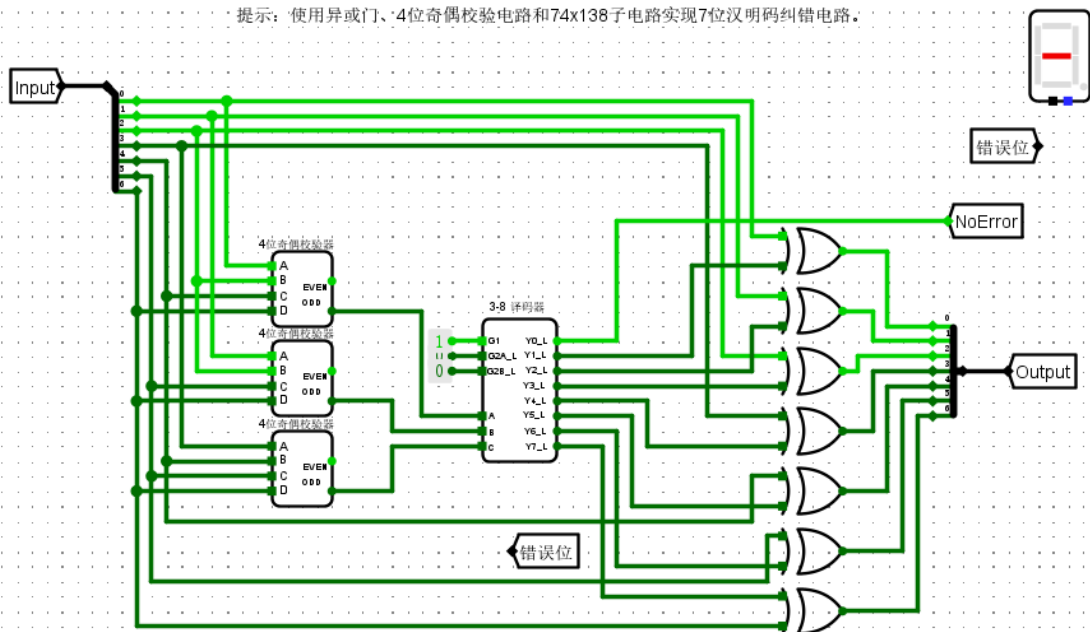
提示：使用异或门、4位奇偶校验电路和74x138子电路实现7位汉明码纠错电路。



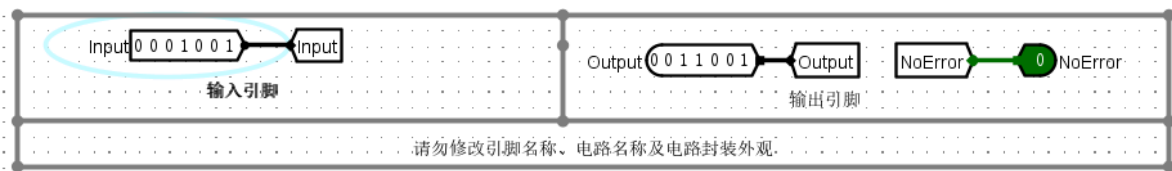
Input=0000111时，无错，Noerror=1.



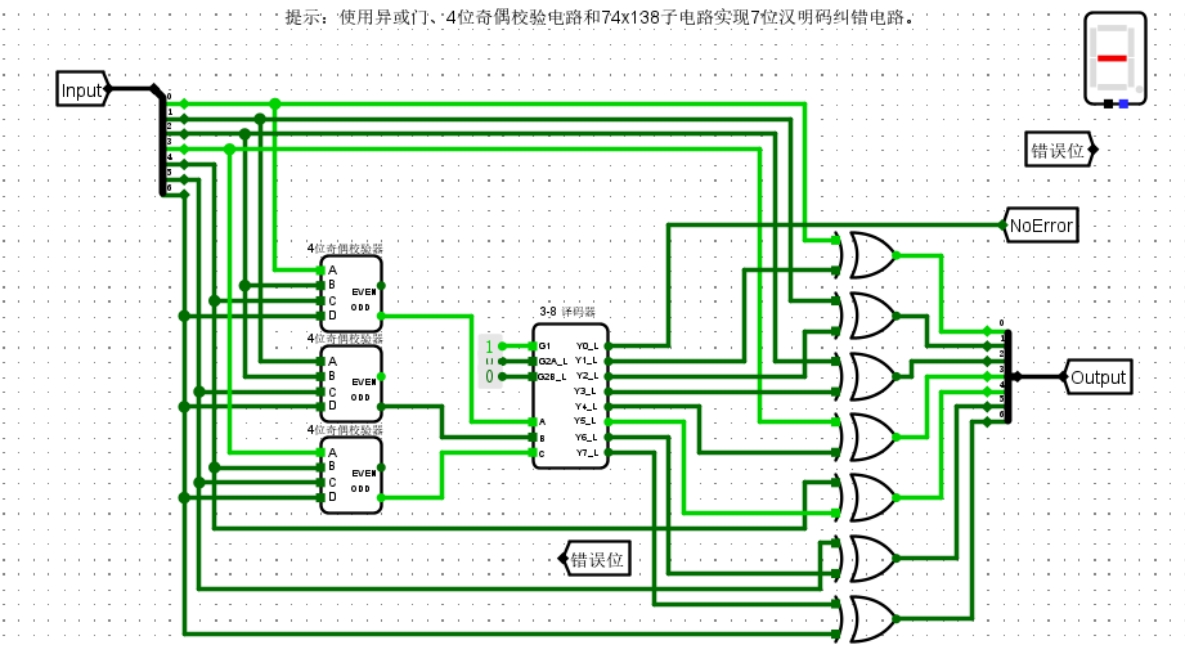
提示：使用异或门、4位奇偶校验电路和74x138子电路实现7位汉明码纠错电路。



Input=0001001，输出纠错后的数值0011001，Noerror=0



提示：使用异或门、4位奇偶校验电路和74x138子电路实现7位汉明码纠错电路。



真值表：

部分如下图：

Input	NoError	Output
00	1	00
01	0	00
02	0	00
03	0	07
04	0	00
05	0	07
06	0	07
07	1	07
08	0	00
09	0	19
0a	0	2a
0b	0	4b
0c	0	4c
0d	0	2d
0e	0	1e
0f	0	07
10	0	00
11	0	19

错误分析

在完成实验的过程中，没有遇到任何错误。

5. 8位桶型移位器实验

实验内容

利用多路选择器实现8位的桶型移位器。

整体方案设计

输入输出引脚：

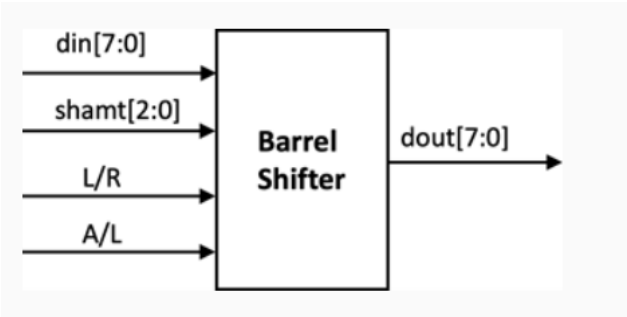
选择端L/R表示左移和右移，选择端A/L表示算术、逻辑和循环移位。（L/R,A/L）赋值为00时表示逻辑右移，赋值为01表示算术右移，赋值位10时表示逻辑左移，赋值位11时表示循环左移。

移位位数shamt为3位，代表要移位的位数

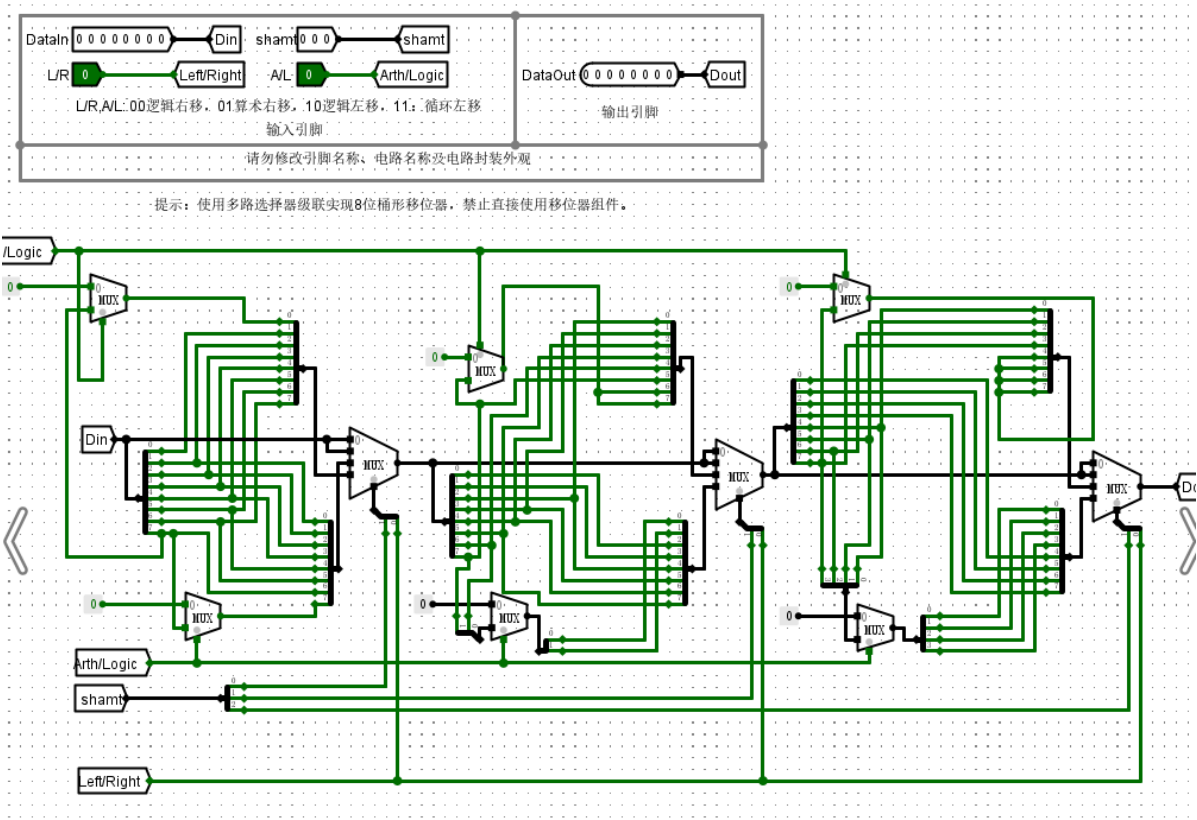
输入数据din和输出数据dout均为8位，代表移位之前和之后的数据。

原理图和电路图

实验原理图如下：

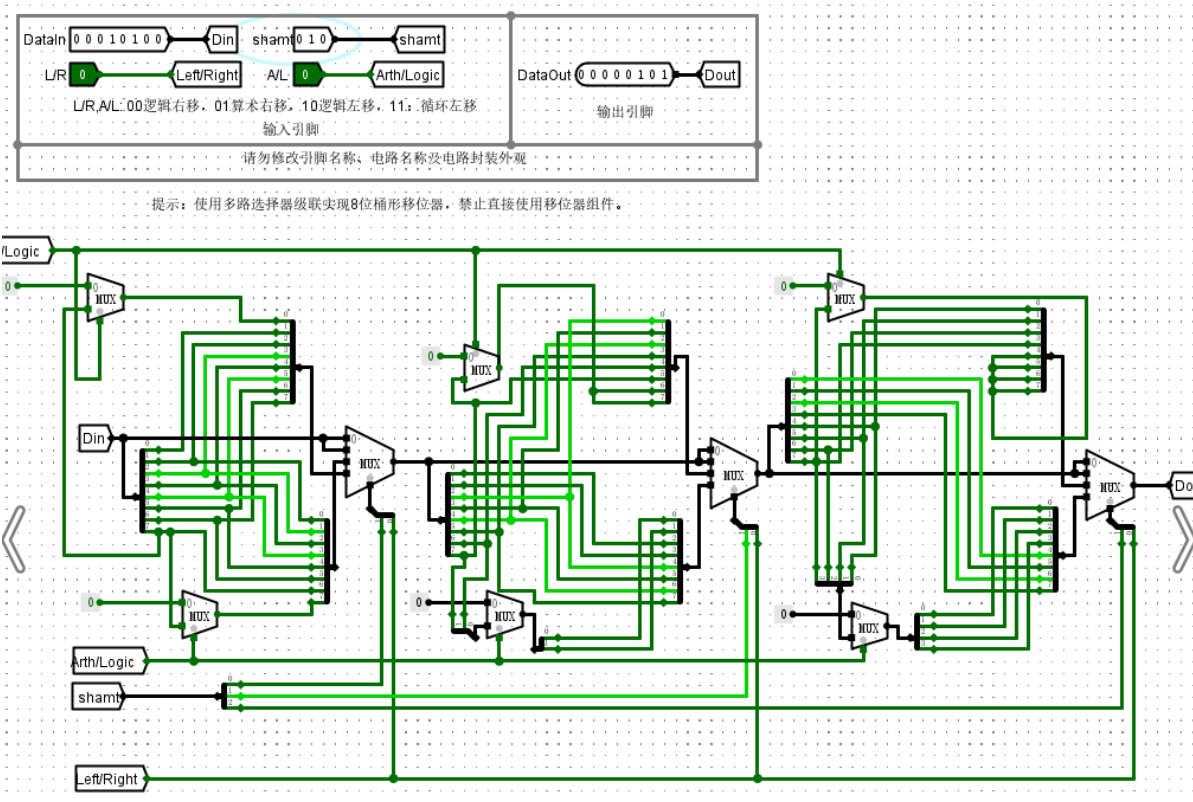


实验电路图如图：

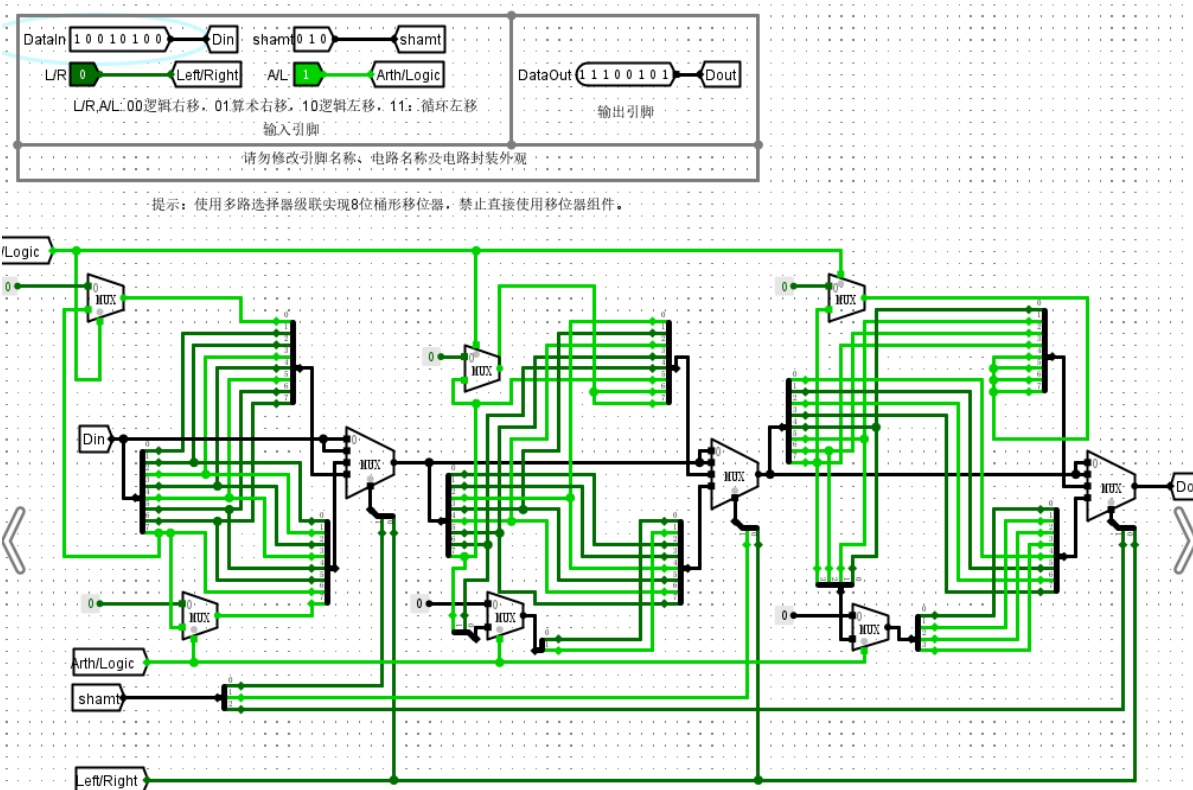


仿真测试图

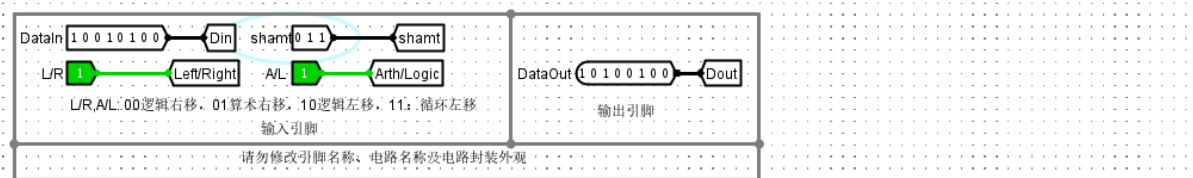
输入=00010100，移位位数为2，00表示逻辑右移，输出为00000101，正确。



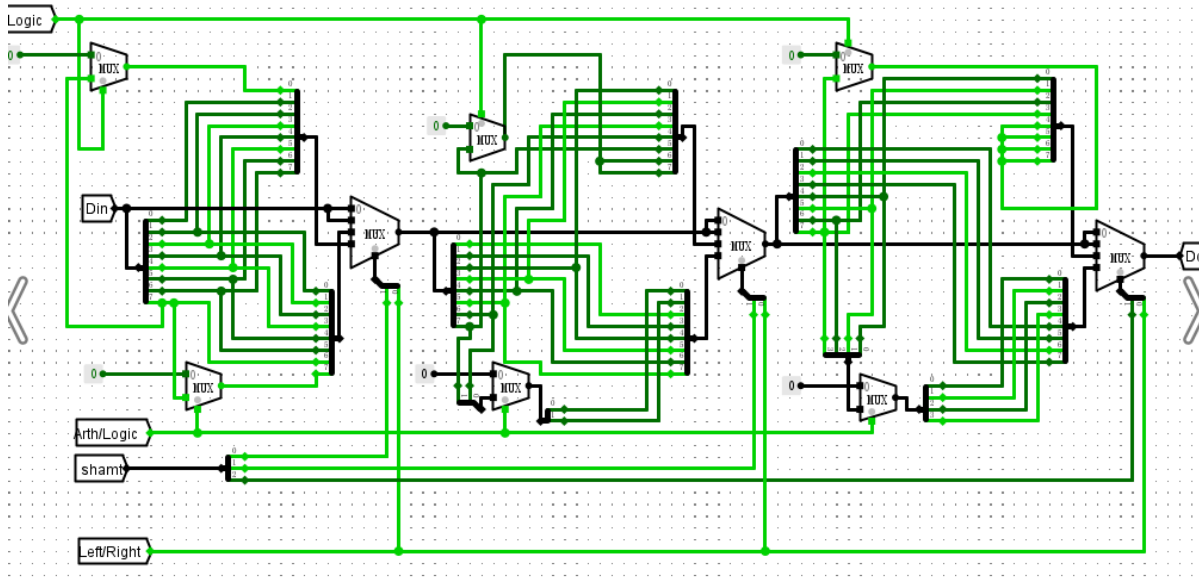
输入=10010100, shamt=010, 01表示算数右移，输出为11100101，正确。



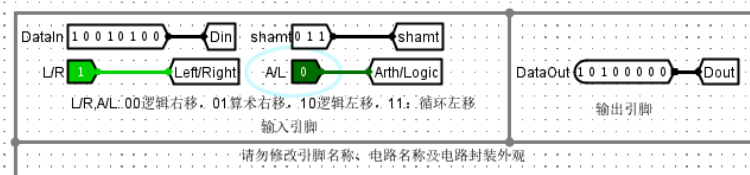
输入=10010100, shamt=011, 11代表循环左移，输出为10100100，正确。



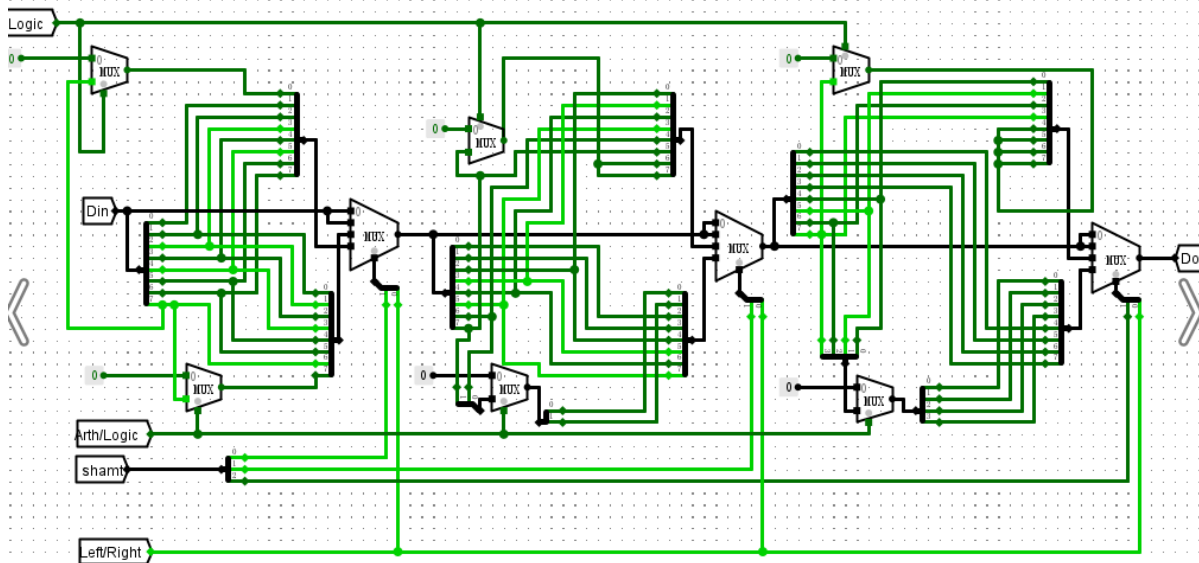
提示：使用多路选择器级联实现8位桶形移位器，禁止直接使用移位器组件。



和上例子完全类似，只是将移位规则改为10，逻辑左移，则输出为1010000，正确。



提示：使用多路选择器级联实现8位桶形移位器，禁止直接使用移位器组件。



真值表的一部分如下：

Din	shamt	LR	AL	Dout
b7	0	0	0	b7
b7	1	0	0	5b
b7	2	0	0	2d
b7	3	0	0	16
b7	4	0	0	0b
b7	5	0	0	05
b7	6	0	0	02
b7	7	0	0	01
b7	0	1	0	b7
b7	1	1	0	6e
b7	2	1	0	dc
b7	3	1	0	b8
b7	4	1	0	70
b7	5	1	0	e0
b7	6	1	0	c0
b7	7	1	0	80
b7	0	0	1	b7
b7	1	0	1	db

错误分析

在完成实验的过程中，没有遇到任何错误。

思考题

T1

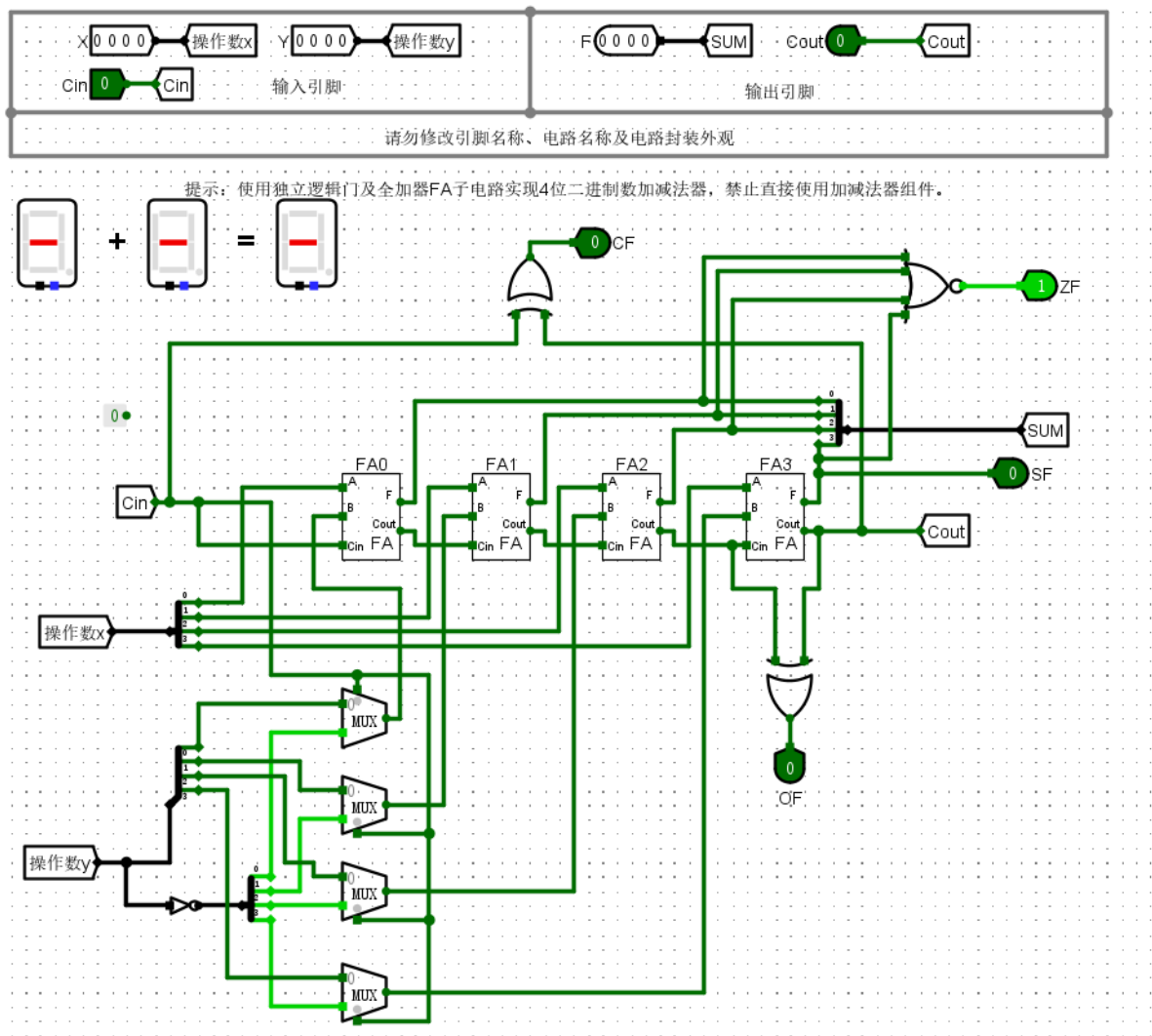
如图所示，实现的思路参考了理论课上讲的四个标志位的生成方法：

$$OF = C_n \oplus C_{n-1}$$

$$SF = F_{n-1}$$

ZF=1当且仅当四位F全为0

$$CF = C_{in} \oplus C_{out}$$



T2

若是无符号数比较大小，则只看CF，CF=0时，大于。

若有符号数比较大小，则看OF和SF，OF=SF时，大于。

T3

思路：以逻辑右移为例

首先将32位的操作数拆分为4个8位二进制数，从低到高位排列为A，B，C，D

将 32 位数据分成4个8位的块。每个8位块可以使用已有的8位桶形移位器，将四个8位移位器按顺序连接成一串。第一个8位移位器处理最低8位(A)，第二个处理接下来的8位(B)，依此类推。

对于移动 4、2、1 位的组合，只使用一串(4 个)8 位桶形移位器即可实现，将shamt的低三位作为8位桶形移位器的 shamt输入。同时，要对桶形移位器做一定的修改，如下：

设计修改八位桶型移位器的输入端，为原有的数字位数和要移入的数字位数，其中，要移入的数字，在右移操作中，为它的高位，如负责移位B的八位桶型移位器，它接受的输入为B和C，其他类似，D作为最高位，它在逻辑右移中直接高位补0即可（如果算术右移就根据符号位补值，也好实现）。这样，就实现了32位数0-7位的移位。

对于移位位数 ≥ 8 但 < 16 的情况，考虑使用两串8位桶形移位器串联(共8个)，即把小于16位的移位分为两步，如现在要移动12位，则先移动6位，再移动六位。对于移位位数 ≥ 16 的情况，使用四串8位桶形移位器串联(共 32个)，即把这个移位操作拆解为4个小于八位的移位操作，即可实现。